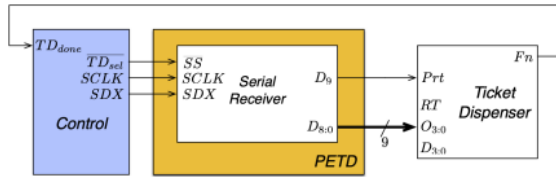


O módulo *Port Expander Ticket Dispenser* é constituído por três blocos principais: *i*) o (*Serial Receiver*); *ii*) o (*Control*); e *iii*) o mecanismo de emissão de bilhetes (*Ticket Dispenser*), de acordo com o diagrama representado na Figura 1. O módulo *Control*, implementado em software, é a entidade produtora que envia os comandos em série, sendo o *Ticket Dispenser* o consumidor final que executa as ações físicas de impressão e corte do bilhete.



**Figura 1:** Diagrama de blocos do módulo *Port Expander Ticket Dispenser*

## 1 Control

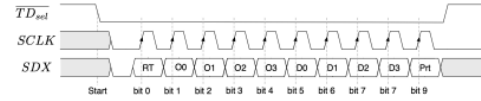
O módulo *Control* constitui o cérebro do sistema da *Ticket Machine*. Implementado em software, é responsável por coordenar as operações de alto nível da máquina, gerir a lógica de negócio (como a seleção de bilhetes e contagem de moedas) e comandar os periféricos de interface. No contexto da emissão de bilhetes, o *Control* atua como a entidade produtora de comandos. Calcula os parâmetros do bilhete pretendido pelo utilizador e encapsula-os numa trama, delegando o envio físico sequencial ao bloco *Serial Emitter* para que o periférico final possa processar a ordem. Composição binária desta trama, como se pode observar na figura 2:

**Bit 0 (*rt*):** Define o tipo de viagem. Assume o valor lógico '1' caso seja um bilhete de ida e volta (*roundTrip*) e '0' caso seja simples (ida).

**Bits 1 a 4 (*org*):** Armazenam o código identificador da estação de origem, permitindo até 16 localizações distintas (4 bits).

**Bits 5 a 8 (*dst*):** Armazenam o código identificador da estação de destino (4 bits).

**Bit 9 (*prt*):** Atua como o sinal ativo de comando de impressão/ativação (*print*). É forçado a nível lógico alto ('1') para dar início ao processo físico no hardware.



**Figura 2:** Protocolo de comunicação com o *Port Expander Ticket Dispenser*

## 2 Serial Receiver

O bloco *Serial Receiver* é o módulo de hardware responsável por realizar a conversão série-paralelo dos dados transmitidos pelo software de (*Control*). Dado que o sistema envia os comandos bit a bit através de uma única linha física de dados, de forma a economizar os pinos do processador, este bloco tem como objetivo capturar sequencialmente esses bits, reconstruir a palavra original paralelamente e disponibilizá-la ao bloco seguinte (*Ticket Dispenser*).

Para efetuar a receção e conversão dos dados, o *Serial Receiver* é constituído internamente por três componentes digitais principais:

**Shift Register:** À medida que os bits de dados são colocados na linha série, o registo desloca o seu conteúdo internamente a cada ciclo de relógio de receção, armazenando temporariamente a informação que vai entrando.

**Bit Counter:** Contador digital do número de bits que já foram recebidos. Quando atinge o valor correspondente ao tamanho predefinido da trama, ele gera um sinal de término (*Counter Flag* ou *Busy*) para avisar a unidade de controlo que a trama está completa.

**Unidade de Controlo (FSM):** Máquina de Estados Finitos que coordena todo o processo de receção. Responsável por detetar a ativação da linha de início, habilitar o contador e o registo de deslocamento nos momentos certos e, no final, gerar um sinal de validação de dados (*DXval* / *Data Transmitted*).

### 3 Ticket Dispenser

O bloco *Ticket Dispenser* é o periférico final do sistema, responsável pela materialização física do processo de compra através da impressão e corte do bilhete.

O Ticket Dispenser recebe os sinais lógicos já expandidos e decodificados pelo *Port Expander Control*, interpreta a informação dos destinos e de tarifário contida nesses sinais e emite o bilhete de transporte configurado pelo utilizador.

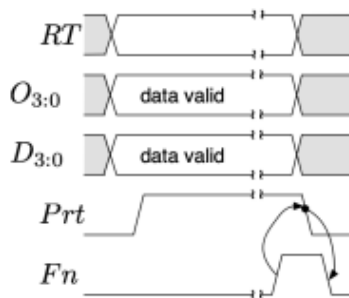
Para que a operação física ocorra de forma correta, o *Ticket Dispenser* é comandado por um barramento paralelo de 10 bits de dados de entrada, cujo mapeamento lógico corresponde exatamente à parametrização definida pelo software, como se pode observar na figura 3:

**Bit de Tipo de Viagem (*rt*):** Posicionado no bit 0, indica se o bilhete é de ida '0' ou de ida e volta '1', alterando a informação textual a ser impressa.

**Bits de Estação de Origem (*O 3..0*):** Alocados entre os bits 1 e 4 (4 bits), fornecem a identificação binária da estação onde a viagem se inicia.

**Bits de Estação de Destino (*D 3..0*):** Alocados entre os bits 5 e 8 (4 bits), fornecem a identificação binária da estação terminal.

**Bit de Ativação de Impressão (*Prt*):** Posicionado no bit 9, este sinal funciona como o sinal de ativação do periférico. Quando este bit transita para o nível lógico '1', o hardware interno do *Ticket Dispenser* inicia imediatamente o ciclo físico de processamento do bilhete. Quando o bilhete é impresso, o *Fn* transita para '1' e o *Prt* volta para nível lógico '0'.



**Figura 3:** Protocolo de comunicação com o mecanismo de *Ticket Dispenser*

### 4 Conclusões

A implementação do módulo *Port Expander Ticket Dispenser* cumpriu com sucesso o objetivo de otimizar os recursos de hardware da *Ticket Machine*. A arquitetura assente na conversão série-paralelo através do *Serial Receiver* provou ser altamente eficiente, reduzindo drasticamente a necessidade de linhas físicas de I/O do processador central para comandar o periférico.

A articulação entre a trama de 10 bits gerada pelo software *Control* e a subsequente expansão de porto permitiu controlar os atuadores com precisão. Adicionalmente, o protocolo de sincronismo baseado em espera ativa (*busy-wait*) garantiu uma comunicação segura, salvaguardando a integridade do sistema eletrónico face aos tempos de resposta mecânicos da impressão e corte do bilhete.

## A VHDL

### A.1 ticketDispenser

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

-- RT bit indicates either is a one-way ticket (1) or a two-way ticket (0)
-- D - Destination Station Code
-- O - Origin Station Code
entity TICKET_DISPENSER is
    port (
        RT, Prt, CollectTicket      : in  STD_LOGIC;
        O, D                        : in  STD_LOGIC_VECTOR(3 downto 0);
        Fn                          : out STD_LOGIC;
        HEX0, HEX1, HEX2, HEX3, HEX4, HEX5 : out STD_LOGIC_VECTOR(7 downto 0)
    );
end entity TICKET_DISPENSER;

architecture arq_TICKET_DISPENSER of TICKET_DISPENSER is

    component DECODERHEX is

        port (
            A      : in  STD_LOGIC_VECTOR(3 downto 0);
            ewr    : in  STD_LOGIC_VECTOR(7 downto 0);
            clear  : in  STD_LOGIC;
            HEX0   : out STD_LOGIC_VECTOR(7 downto 0)
        );
    end component DECODERHEX;

    signal RTtoBDC : STD_LOGIC_VECTOR(3 downto 0);

begin

    -- Activate Fn when client retrieves the printed ticket
    Fn <= CollectTicket;

    -- Convert RT bit to BDC
    RTtoBDC <= "0000" when RT = '1' else "0001";

    -- Using the six 7 segment displays available on the MAX10 Lite Board
    -- produce a valid array of numbers and letters which the client can understand
    -- Valid format example: [b0oid4]
    -- Means: "bilhete de ida e volta, da estação de origem com o código (0001) para a estação de destino
    -- com o código (0100)
    -- Implementation on the six 7 displays segment, facing the board, starting from the right to the
    -- left:
    U0: component DECODERHEX
    port map (
        A      => D,
        ewr    => "11111111",
        clear  => not Prt,
        HEX0   => HEX4
    );

    U1: component DECODERHEX
    port map (
        -- Always d in BDC
        A      => "1101",
        ewr    => "11111111",
        clear  => '0',
        HEX0   => HEX5
    );
end architecture arq_TICKET_DISPENSER;

```

```
);

U2: component DECODERHEX
port map (
    A      => 0,
    ewr    => "11111111",
    clear  => not Prt,
    HEX0   => HEX2
);

U3: component DECODERHEX
port map (
    A      => "0000",
    -- A value assignment is redundant here, since the value in ewr will overwrite it, because its
    --   different from "11111111"
    -- Always 0, which has no representation in BDC
    ewr    => "10100011",
    clear  => '0',
    HEX0   => HEX3
);

U4: component DECODERHEX
port map (
    -- RT bit in BDC
    A      => RTtoBDC,
    ewr    => "11111111",
    clear  => not Prt,
    HEX0   => HEX0
);

U5: component DECODERHEX
port map (
    -- Always b in BDC
    A      => "1011",
    ewr    => "10110110",
    clear  => '0',
    HEX0   => HEX1
);

end architecture arq_TICKET_DISPENSER;
```

## A.2 PETD

```
library ieee;
use ieee.std_logic_1164.all;

entity PETD is
    port (
        SDX, CLK, SS, RESET : in  std_logic;
        D                    : out std_logic_vector(7 downto 0);
        Prt, Rt              : out std_logic
    );
end entity PETD;

architecture logicFunction of PETD is

    component SerialReceiver is
        port (
            SDX, CLK, SS, RESET : in  std_logic;
            Q                    : out std_logic_vector(9 downto 0)
        );
    end component SerialReceiver;

begin

serial: component SerialReceiver port map(
    SDX      => SDX,
    CLK      => CLK,
    SS       => SS,
    RESET    => RESET,
    Q(1)     => D(7),
    Q(2)     => D(6),
    Q(3)     => D(5),
    Q(4)     => D(4),
    Q(5)     => D(3),
    Q(6)     => D(2),
    Q(7)     => D(1),
    Q(8)     => D(0),
    Q(0)     => Prt,
    Q(9)     => Rt
);
end architecture logicFunction;
```

## B *Kotlin*

### B.1 SerialEmitter

Algoritmo 1: SerialEmitter

```
object SerialEmitter {  
    enum class Peripheral {LCD, TICKET}  
  
    private const val SDX_MASK = 0b00000001  
    private const val SCLK_MASK = 0b00000010  
    private const val SSLCD_MASK = 0b00000100  
    private const val SSTD_MASK = 0b00001000  
  
    fun init() {  
        HAL.init()  
        HAL.clrBits(SDX_MASK or SCLK_MASK or SSLCD_MASK or SSTD_MASK)  
    }  
  
    fun send(addr: Peripheral, data: Int, size: Int) {  
        var _data = data  
  
        HAL.clrBits(SCLK_MASK)  
  
        if (addr == Peripheral.LCD) {  
            HAL.clrBits(SSLCD_MASK)  
        } else if (addr == Peripheral.TICKET) {  
            HAL.clrBits(SSTD_MASK)  
        }  
  
        for (i in 0 until size) {  
            HAL.writeBits(SDX_MASK, _data)  
            _data = _data.shr(1)  
            Thread.sleep(0, 40) // deixa SDX estabilizar  
            HAL.setBits(SCLK_MASK)  
            Thread.sleep(0, 40) // clock high time  
            HAL.clrBits(SCLK_MASK)  
        }  
  
        if (addr == Peripheral.LCD) {  
            HAL.setBits(SSLCD_MASK) // SS=1  
            Thread.sleep(1) // tempo suficiente  
            HAL.setBits(SCLK_MASK) // pulso de clock extra para HoldRegister capturar  
            HAL.clrBits(SCLK_MASK)  
            Thread.sleep(1)  
            HAL.clrBits(SSLCD_MASK)  
        } else if (addr == Peripheral.TICKET) {  
            HAL.setBits(SSTD_MASK) // SS=1  
            Thread.sleep(1) // tempo suficiente  
            HAL.setBits(SCLK_MASK) // pulso de clock extra para HoldRegister capturar  
            HAL.clrBits(SCLK_MASK)  
            Thread.sleep(1)  
            HAL.clrBits(SSTD_MASK)  
        }  
    }  
}
```

## B.2 TicketDispenser

### Algoritmo 2: TicketDispenser

```
object TicketDispenser{
  private const val FN_MASK = 0b00010000

  fun init( ){
  }

  fun activatePrintingTicket ( roundTrip : Boolean , origin : Int , destination : Int ) {
    val rt = if (roundTrip) 1 else 0

    val org = (origin and 0xF) shl 1 // bits 1..4
    val dst = (destination and 0xF) shl 5 // bits 5..8
    val prt = 1 shl 9 // bit 9

    val frame = rt or org or dst or prt

    SerialEmitter.send(SerialEmitter.Peripheral.TICKET, frame, 10)

    while (!HAL.isBit(FN_MASK)) {
      Thread.sleep(10)
    }
  }
}
```